LSIS-AFS Interoperability Testing

Overview

This document defines the interoperability testing strategy for LSIS-AFS reference implementations. Interoperability ensures that different implementations can exchange signals and data correctly, validating compliance with the LSIS-AFS specification.

Interoperability Goals

1. **Signal Compatibility**: Signals generated by one implementation can be decoded by another
2. **Data Fidelity**: Navigation data survives the encode-decode cycle across implementations
3. **Specification Compliance**: All implementations produce identical outputs for identical inputs
4. **Cross-Platform Validation**: Implementations in different languages produce compatible results

Test Levels

Level 1: Code Generation Interoperability

Objective: Verify all implementations generate identical spreading codes

Test Procedure: 1. Each implementation generates codes for PRN 1-210 2. Export codes in standardized format (hexadecimal) 3. Compare outputs byte-by-byte 4. Validate against Annex3 reference codes

Pass Criteria: - 100% match with reference codes from Annex3 - Identical outputs across all implementations - All 210 PRN codes validated

Test Vectors:

```
PRN 1 Gold Code (first 32 chips): 0x7A8B4C2D...
PRN 1 Weil Primary (first 32 chips): 0x3F5E9A1B...
PRN 1 Weil Tertiary (first 32 chips): 0x8C4D2E1F...
```

Level 2: Encoding Interoperability

Objective: Verify all implementations produce identical encoded frames

Test Procedure: 1. Define standard test messages with known data 2. Each implementation encodes the same message 3. Export encoded frames in standardized format 4. Compare frame structures bit-by-bit

Pass Criteria: - Identical sync patterns (68 symbols) - Identical BCH-encoded Subframe 1 - Identical LDPC-encoded Subframes 2, 3, 4 - Identical interleaving patterns - Frame structure matches specification exactly

Test Messages: - **Test Message 1**: All zeros (baseline) - **Test Message 2**: All ones (inverse baseline) - **Test Message 3**: Alternating pattern (0xAA…) - **Test Message 4**: Known ephemeris data - **Test Message 5**: Random data with known seed

Level 3: Signal Generation Interoperability

Objective: Verify all implementations generate compatible I/Q signals

Test Procedure: 1. Use identical navigation data and PRN code 2. Each implementation generates 12-second I/Q signal 3. Export signals in standardized format (binary, CSV) 4. Cross-decode: Implementation A signal → Implementation B decoder

Pass Criteria: - Signals have identical chip rates (1.023 / 5.115 Mchip/s) - Signals have identical symbol rates (500 symbols/s for AFS-I) - Code synchronization is identical - Cross-decoding recovers original data with BER < 10

Signal Parameters: - Sample rate: 10.23 MHz (common baseline) - Duration: 12 seconds (one frame) - Format: Complex float32 (I/Q interleaved) - PRN: Use test codes from Table 11

Level 4: Decoding Interoperability

Objective: Verify all implementations can decode each other's signals

Test Procedure: 1. Implementation A generates signal with known data 2. Implementation B decodes the signal 3. Compare decoded data with original input 4. Repeat for all implementation pairs

Pass Criteria: - Frame synchronization successful (>99% detection) - All subframes decoded correctly - CRC validation passes - Decoded data matches original input exactly

Test Matrix:

```
         Decoder A  Decoder B  Decoder C
Signal A
Signal B
Signal C
```

Level 5: Message Parsing Interoperability

Objective: Verify all implementations extract identical navigation data

Test Procedure: 1. Use common decoded frame data 2. Each implementation parses all subframes 3. Export parsed fields in standardized format (JSON) 4. Compare extracted values field-by-field

Pass Criteria: - FID and TOI extracted correctly - Week Number (WN) and ITOW match exactly - Clock & Ephemeris fields match exactly - Time of Transmission calculated identically - All message type fields parsed correctly

Test Fields: - Subframe 1: FID, TOI - Subframe 2: WN, ITOW, CED parameters, Health status - Subframe 3: Type field, almanac data - Subframe 4: Type field, network access data

Standardized Test Vector Format

Code Export Format

Filename: `codes_prn{N}.hex`

Format:

```
# LSIS-AFS Code Test Vector
# PRN: {N}
# Generated: {ISO-8601 timestamp}

[GOLD_CODE]
length: 2046
hex: 7A8B4C2D...

[WEIL_PRIMARY]
length: 10230
hex: 3F5E9A1B...

[WEIL_TERTIARY]
length: 1500
hex: 8C4D2E1F...

[SECONDARY_S0]
hex: E
```

```
[SECONDARY_S1]
hex: 7

[SECONDARY_S2]
hex: B

[SECONDARY_S3]
hex: D
```

Frame Export Format

Filename: `frame_{message_id}.bin`

Format: Binary file with metadata header

```
Header (64 bytes):
  - Magic: "LSISAFS\0" (8 bytes)
  - Version: 1 (4 bytes)
  - Frame length: 6000 (4 bytes)
  - PRN: {N} (4 bytes)
  - Timestamp: Unix epoch (8 bytes)
  - Reserved: (36 bytes)

Data (6000 symbols):
  - Each symbol: 1 byte (0x00 or 0x01)
```

Signal Export Format

Filename: `signal_{prn}_{duration}s.iq`

Format: Binary I/Q samples

```
Header (128 bytes):
  - Magic: "LSISIQ\0\0" (8 bytes)
  - Version: 1 (4 bytes)
  - Sample rate: float64 (8 bytes)
  - Duration: float64 seconds (8 bytes)
  - PRN: uint32 (4 bytes)
  - Sample format: "float32" (16 bytes)
  - Reserved: (80 bytes)

Data:
  - Samples: float32 I/Q interleaved [I0, Q0, I1, Q1, ...]
```

Parsed Data Export Format

Filename: `parsed_{frame_id}.json`

Format: JSON

```
{
  "version": "1.0",
  "timestamp": "2026-03-07T12:00:00Z",
  "frame_id": "test_001",
  "subframe1": {
    "fid": 0,
    "toi": 42
```

```
  },
  "subframe2": {
    "wn": 1234,
    "itow": 256,
    "ced": {
      "af0": 1.23e-9,
      "af1": 4.56e-12,
      "...": "..."
    },
    "health": 0,
    "time_conversions": {}
  },
  "subframe3": {
    "type": 1,
    "data": {}
  },
  "subframe4": {
    "type": 1,
    "data": {}
  },
  "time_of_transmission": 1234567890.123
}
```


Interoperability Test Suite

Test Case 1: Baseline Validation

Purpose: Verify basic functionality with simplest inputs

Inputs: - PRN: 1 (first test code from Table 11) - Message: All zeros - Duration: 12 seconds

Expected Outputs: - Codes match Annex3 reference - Frame structure per Figure 9 - Signal decodable with BER < 10

Test Case 2: All Test Codes

Purpose: Verify all 12 interim test codes work

Inputs: - PRN: 1-12 (Table 11) - Message: Standard test message - Duration: 12 seconds each

Expected Outputs: - All codes generate correctly - All signals decode correctly - Cross-implementation compatibility

Test Case 3: Message Type Coverage

Purpose: Verify all message types encode/decode correctly

Inputs: - All Subframe 3 message types - All Subframe 4 message types - Various data patterns

Expected Outputs: - All message types parse correctly - Type fields extracted correctly - Data fields match specification

Test Case 4: Boundary Conditions

Purpose: Test edge cases and limits

Inputs: - Maximum field values (WN=8191, ITOW=503, TOI=99) - Minimum field values (all zeros) - PRN 210 (highest code number)

Expected Outputs: - No overflow or underflow - Correct handling of edge values - Proper validation of ranges

Test Case 5: Error Conditions

Purpose: Verify error handling and CRC validation

Inputs: - Corrupted frames (bit flips) - Invalid CRC - Out-of-range field values

Expected Outputs: - CRC validation detects errors - Decoders handle errors gracefully - Error rates match specification

Cross-Implementation Testing Protocol

Phase 1: Setup

1. Each team provides their implementation
2. Agree on test vector formats
3. Generate reference test vectors
4. Distribute test vectors to all teams

Phase 2: Independent Testing

1. Each team runs test suite independently
2. Generate outputs in standardized formats
3. Document any deviations or issues
4. Submit results to coordination team

Phase 3: Cross-Validation

1. Exchange generated test vectors
2. Each implementation tests against others' outputs
3. Document compatibility matrix
4. Identify and resolve discrepancies

Phase 4: Compliance Verification

1. Compare all outputs against specification
2. Validate against Annex3 reference codes
3. Measure performance metrics
4. Generate compliance report

Interoperability Metrics

Code Generation Metrics

- **Match Rate**: Percentage of codes matching reference (target: 100%)
- **Generation Time**: Time to generate all 210 PRN codes (target: < 210 seconds)

Encoding Metrics

- **Frame Correctness**: Percentage of frames matching reference (target: 100%)
- **Encoding Throughput**: Frames per second (target: > 10 fps)

Signal Generation Metrics

- **Signal Compatibility**: Percentage of cross-decode successes (target: 100%)
- **BER Performance**: Bit error rate at SNR = 0 dB (target: < 10)

Decoding Metrics

- **Frame Detection Rate**: Percentage of frames detected (target: > 99%)
- **Decode Success Rate**: Percentage of frames decoded correctly (target: > 99%)
- **Decoding Throughput**: Frames per second (target: > 1 fps)

Parsing Metrics

- **Field Accuracy**: Percentage of fields parsed correctly (target: 100%)
- **Time Accuracy**: ToT calculation error (target: < 1 microsecond)

Compliance Checklist

Specification Compliance

All codes match Annex3 references
Frame structure matches Figure 9
Bit allocations match Tables 14, 18, 19, 20
Symbol counts: 68 + 52 + 5880 = 6000
Frame duration: 12 seconds
Chip rates: 1.023 / 5.115 Mchip/s
Symbol rate: 500 symbols/s for AFS-I

Interoperability Compliance

Codes identical across implementations
Frames identical across implementations
Signals cross-decode successfully
Parsed data matches across implementations
All test vectors pass validation

Performance Compliance

Code generation < 1 second per PRN
Frame encoding < 100ms per frame
Frame decoding < 1 second per frame
BER < 10 at SNR > 0 dB

Test Vector Repository Structure

```
test-vectors/
  codes/
     prn001.hex
     prn002.hex
     ...
  frames/
     test_message_1.bin
     test_message_2.bin
     ...
  signals/
     prn001_12s.iq
     prn002_12s.iq
     ...
  parsed/
     frame_001.json
     frame_002.json
```

```
    ...
reference/
    annex3_codes/
    specification_examples/
    validation_data/
```

Reporting Format

Interoperability Test Report

Template:

```
# LSIS-AFS Interoperability Test Report

## Implementation Details
- Name: {Implementation Name}
- Version: {Version}
- Language: {Language}
- Date: {ISO-8601}

## Test Results Summary
- Total Tests: {N}
- Passed: {N}
- Failed: {N}
- Pass Rate: {X}%

## Code Generation Results
- PRN Codes Tested: 210
- Matches with Reference: {N}/210
- Match Rate: {X}%

## Encoding Results
- Test Frames: {N}
- Correct Frames: {N}
- Correctness Rate: {X}%

## Signal Generation Results
- Signals Generated: {N}
- Cross-Decode Success: {N}
- Compatibility Rate: {X}%

## Decoding Results
- Signals Decoded: {N}
- Successful Decodes: {N}
- Success Rate: {X}%

## Issues Identified
1. {Issue description}
2. {Issue description}

## Compliance Status
- Specification Compliant: {Yes/No}
- Interoperable: {Yes/No}
- Performance Compliant: {Yes/No}
```

Continuous Integration

Automated Testing

1. Run test suite on every commit
2. Generate test vectors automatically
3. Compare against reference vectors
4. Report deviations immediately

Regression Testing

1. Maintain test vector archive
2. Test new versions against historical vectors
3. Ensure backward compatibility
4. Track performance over time

Conclusion

Interoperability testing ensures that LSIS-AFS implementations can work together seamlessly. By following this testing strategy and using standardized test vectors, teams can validate their implementations and ensure compliance with the specification.

The key to successful interoperability is: 1. **Standardized formats** for exchanging data 2. **Comprehensive test vectors** covering all scenarios 3. **Cross-validation** between implementations 4. **Continuous testing** throughout development